IA08: Security, Privacy and Ethical Review

Video: {{VIDEO_URL - add after recording}} · **Diagram:** See attached

IA08_threat_model (exported from draw.io)

Threat 1 - Cross-site request forgery (CSRF) on checkout

Field	Detl
Threat	A malicious website tricks a logged-in customer's browser into POSTing to /checkout.php , placing an order the customer did not intend.
Likelihood	Medium - any logged-in user visiting a malicious page while their session is active.
Impact	Unwanted order placed under the customer's account; customer charged or food wasted.
Current control	csrf_token generated per session, stored in $_SESSION['csrf_token'] , embedded in the form, compared with hash_equals() on POST. A cross-site request cannot read the token (same-origin policy).
Remning gap	Session cookie does not have SameSite set explicitly. Modern browsers default to Lax, but it should be Strict : session_set_cookie_params(['samesite' => 'Strict']) .
Residual risk	Low after the explicit SameSite=Strict fix.

Threat 2 - Session hijacking

Field	Detl
Threat	An attacker intercepts the session cookie (e.g. on shared Wi-Fi) and uses it to place orders, view order history o access the admin dashboard.
Likelihood	Low on HTTPS; Medium on pln HTTP (current state).

Impact	High - customer addresses, spending history and staff dashboard accessible.
Current control	None for transport. password_hash() protects stored passwords so a DB leak does not give plntext passwords.
Remning gap	No HTTPS, no Secure or HttpOnly cookie flags. Three lines to fix: add flags to session_set_cookie_params() ; HTTP to HTTPS in web server config.
Residual risk	High until HTTPS is deployed. This is the most critical outstanding risk.

Threat 3 - Broken access control (any staff = any action)

Field	**Detl**
Threat	A kitchen staff member sets any order to cancelled, or any staff member views revenue totals and all customer delivery addresses.
Likelihood	High - admin/update_order.php checks only that the user is staff; it does not check which statuses that role may
Impact	Medium - a disgruntled or mistaken kitchen worker could cancel live orders; all staff see addresses and revenue client explicitly sd only owner/manager should see financial reports.
Current control	None beyond the staff login check.
Remning gap	BUG-01 and BUG-02: add a role-to-action permission map, enforce it in update_order.php and admin/index.ph return 403 if the role lacks permission.
Residual risk	High until role guards are added.

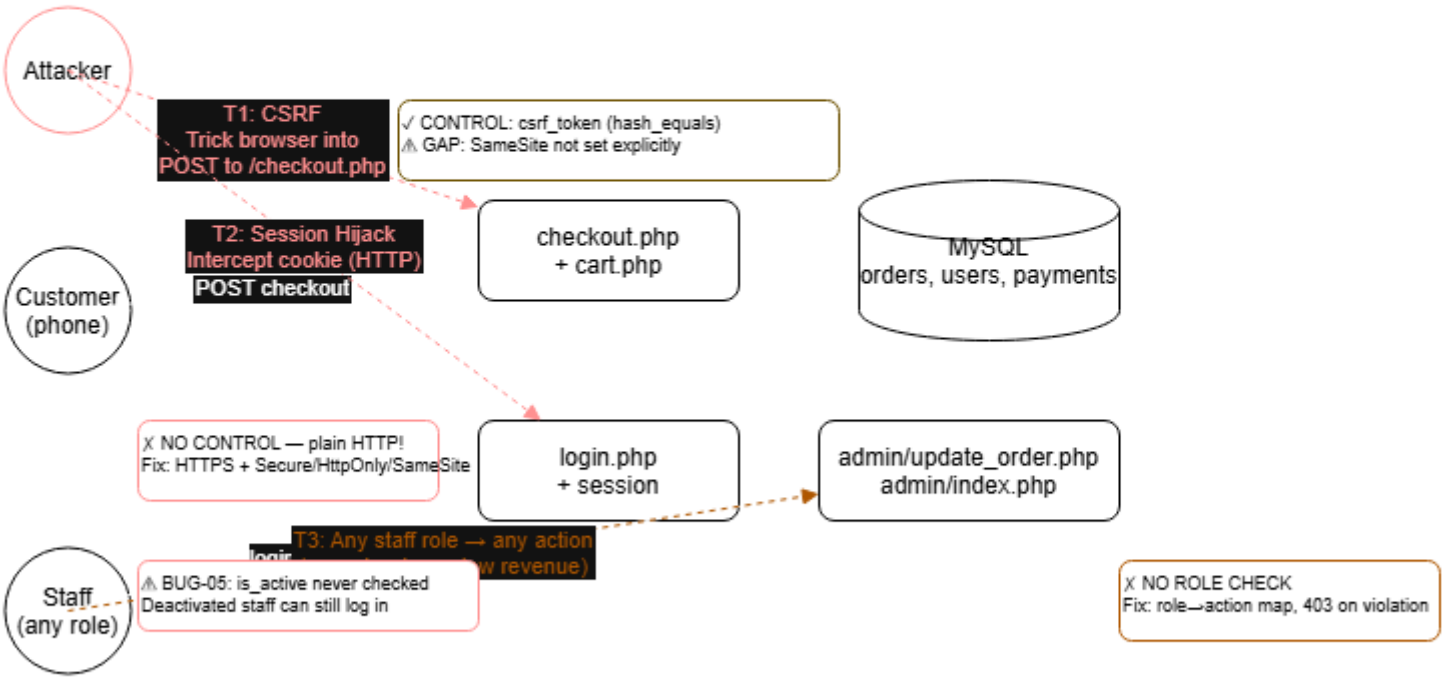
Ethical concern: deactivated staff can log in (BUG-05) is_active exists as a column in users but is never checked in login.php . A staff member who has left the restaurant can still log in, view all orders and update statuses. This is both a data privacy flure (they

should not see customer addresses after leaving) and an access control flure. Fix: add AND is_active = 1 to the login query.

Two prioritised security improvements

Priority 1 - HTTPS and session cookie hardening. Reason: pln HTTP means every session cookie on every page load is interceptable - this affects all users at once. Fix: get a TLS certificate (Let's Encrypt, free), redirect port 80 to 443, add Secure / HttpOnly / SameSite=Strict to session_set_cookie_params() in includes/functions.php . Cost: low.

Priority 2 - Role-based action guards. Reason: any staff login can perform any staff action. Fix: define a $role permissions map and check it in update order.php and _ p _ admin/index.php , returning 403 on violation. Cost: a few hours.


Attacker
T1: CSRF
Trick browser into
POST to /checkout.php
✓ CONTROL: csrf_token (hash_equals)
⚠ GAP: SameSite not set explicitly
checkout.php
+ cart.php
MySQL
orders, users, payments
T2: Session Hijack
Intercept cookie (HTTP)
POST checkout
Customer
(phone)
✗ NO CONTROL — plain HTTP!
Fix: HTTPS + Secure/HttpOnly/SameSite
login.php
+ session
admin/update_order.php
admin/index.php
T3: Any staff role → any action
Staff
(any role)
⚠ BUG-05: is_active never checked
Deactivated staff can still log in
✗ NO ROLE CHECK
Fix: role→action map, 403 on violation